



CYBERSTER

Cyber Security Internship Program

INTERNSHIP WEEKLY REPORT

Week 2 — Network Enumeration & Service Vulnerability Discovery (Red Team Track)

Topic: Advanced Nmap Scanning, Service Fingerprinting & Banner Grabbing, Nmap Scripting Engine (NSE) Vulnerability Discovery, and Firewall/IDS Evasion Techniques — performed against Metasploitable2, an intentionally vulnerable, legally authorized practice target

Student Name: Umar Farooq

Batch#: 2

Instructor: Umar Niaz

Submission Date: July 2, 2026

Contents

Executive Summary	4
Learning Objectives.....	6
Environment Setup	7
Operating System.....	7
Virtual Machine Configuration	7
Network Configuration	7
Lab Architecture.....	7
Installed Tools	8
Task 1: Advanced Network Scanning with Nmap	9
Objective	9
Theory / Background.....	9
Tools Used.....	9
Procedure.....	9
Commands Used.....	10
Output / Results.....	10
Screenshots & Evidence	11
Observations.....	16
Key Learning Points	16
Security Relevance	17
Task 2: Service Fingerprinting & Banner Grabbing	18
Objective	18
Theory / Background.....	18
Tools Used.....	18
Procedure.....	18
Commands Used.....	18
Output / Results.....	19
Screenshots & Evidence	20
Observations.....	24
Key Learning Points	24
Security Relevance	25
Task 3: Leveraging Nmap Scripting Engine (NSE).....	26
Objective	26
Theory / Background.....	26

Tools Used.....	26
Procedure.....	26
Commands Used.....	27
Output / Results.....	27
Screenshots & Evidence	28
Observations.....	29
Key Learning Points	29
Security Relevance	30
Task 4: Stealth & Firewall Evasion Techniques	31
Objective	31
Theory / Background.....	31
Tools Used.....	31
Procedure.....	31
Commands Used.....	32
Output / Results.....	32
Screenshots & Evidence	33
Observations.....	34
Key Learning Points	34
Security Relevance	35
Consolidated Service & Vulnerability Table.....	36
Evasion Results Summary.....	37
Actionable Insights.....	37
Technical Concepts Learned.....	39
TCP Three-Way Handshake.....	39
Port States: Open, Closed, and Filtered.....	39
Service Fingerprinting.....	39
Null Session (SMB)	39
Nmap Scripting Engine (NSE) Categories.....	39
CVE Mapping and Verification.....	39
Packet Fragmentation	39
Decoy Scanning	40
Source Port Spoofing	40
Intrusion Detection System (IDS) Alerting.....	40
Challenges Faced.....	41

Best Practices	42
Weekly Summary Table	43
Conclusion	44
References	45
Appendix	46
A. Raw Command Outputs Retained	46
B. Screenshot Index	46
C. Submission Checklist	46

Executive Summary

During the second week of my Cyber Security Internship at Cyberster, on the Red Team track, I moved from the purely passive information-gathering work of Week 1 into active, packet-level engagement with a target host. Where Week 1 built a picture of an organization's external footprint without a single packet touching its infrastructure, Week 2 required me to actually send traffic to a machine, interpret its responses, and reason about what those responses reveal about the services running behind each open port. I carried out this week's entire task set against Metasploitable2, an intentionally vulnerable Linux virtual machine maintained by Rapid7 specifically for this kind of hands-on practice, running inside an isolated Host-Only VirtualBox network alongside my Kali Linux attack workstation.

The week was organized around four practical tasks that together mirror the scanning and enumeration phase of a real penetration testing engagement. I began with comparative Nmap scanning, running TCP Connect and SYN Stealth scans against the full port range, a dedicated UDP sweep of the top one hundred ports, and a full pass through Nmap's six timing templates to observe the direct trade-off between scan speed and reliability. I then moved into service fingerprinting, using Nmap's version-detection and aggressive-scan modes alongside manual banner grabbing with Netcat and a full Samba enumeration pass with enum4linux once port 445 was confirmed open. The third task introduced the Nmap Scripting Engine, where I ran discovery and safe-category scripts for lightweight enumeration before executing the vulnerability category outright and mapping its findings back to specific, named CVE identifiers. The final task shifted focus to evasion, where I practiced packet fragmentation, decoy scanning, and source-port spoofing to understand how a stealthier scan profile changes what a defensive control sees.

By the end of the week I had a complete port and service inventory for the target, a documented set of exploitable vulnerabilities each tied back to a specific CVE, and a firsthand comparison of which evasion techniques meaningfully changed my scan's visibility versus which ones only changed its syntax without a firewall or IDS present to actually test against. Just as importantly, I built a working understanding of why a stealth SYN scan is the default choice in professional red-teaming, why a service's version string is the single most valuable piece of information a scan can return, and why a scan that is loud and fast is a scan that gets blocked before it finishes. I close Week 2 with a repeatable enumeration workflow that extends directly from the reconnaissance foundation I built in Week 1, and with a clear sense of how the two phases connect: reconnaissance tells you a host exists, and enumeration tells you exactly how to get into it.

Learning Objectives

By the end of Week 2, I was expected to be able to:

Explain the mechanics of the TCP three-way handshake and articulate, in concrete technical terms, why a SYN Stealth scan is less detectable than a full TCP Connect scan even though both ultimately reveal the same open ports.

Perform and interpret UDP scanning, understanding why UDP enumeration is inherently slower and less reliable than TCP enumeration due to the way closed UDP ports are identified.

Use Nmap's timing templates deliberately, selecting a specific template to balance scan speed against the likelihood of detection or dropped packets, rather than defaulting to whatever template happens to be fastest.

Fingerprint the exact version of a network service using both automated tooling and manual verification, and treat that version string as the starting point for vulnerability research rather than an end in itself.

Enumerate SMB shares, users, and operating system details from a target host using enum4linux, and understand what a null-session SMB misconfiguration actually exposes to an unauthenticated attacker.

Use the Nmap Scripting Engine across multiple script categories, distinguishing between scripts that passively enrich a scan's output and scripts that actively probe for exploitable conditions.

Map NSE vulnerability-script output to specific, named CVE identifiers and independently verify each mapping against an external vulnerability database rather than trusting the script's own labeling.

Apply packet fragmentation, decoy scanning, and source-port spoofing, and critically evaluate which of these techniques would plausibly evade a real intrusion detection system versus which only alter the scan's outward appearance.

Produce a documented, evidence-backed scanning and enumeration report structured well enough to be handed to a supervisor or client exactly as it stands.

Environment Setup

Operating System

All attacking-side work for Week 2 was carried out from the same Kali Linux (rolling release) installation used during Week 1, accessed through a terminal multiplexer session shown throughout this report's screenshots as `mutex@kali`. The working directory for this week's evidence and output files was `~/Desktop/Cyberster/Red Team/week2`, kept separate from the Week 1 directory so that this week's raw scan files, screenshots, and NSE output could not be confused with the previous week's reconnaissance artifacts.

Virtual Machine Configuration

Unlike Week 1, this week's task set required a second, dedicated virtual machine to actually scan, since HackThisSite is a live production platform not intended for active port scanning or vulnerability-scripting traffic. I imported Rapid7's official Metasploitable2 OVA into VirtualBox, allocated it 512 MB of RAM and a single virtual CPU core in line with the image's own minimum requirements, and left its virtual disk in the default pre-built state so that the intentionally vulnerable service set the image ships with remained unmodified. I logged into the machine once, using the published `msfadmin/msfadmin` credentials, purely to confirm it had booted correctly and to read its assigned IP address from `ifconfig` before returning to Kali for the remainder of the week's work.

Network Configuration

Both the Kali attacker machine and the Metasploitable2 target were placed on the same VirtualBox network segment, with the target resolving to 10.0.2.3 and the attacker workstation resolving to 10.0.2.15 for the duration of the week's testing. I deliberately kept this segment isolated from any untrusted network, since the target's own web service banner explicitly warns 'Never expose this VM to an untrusted network!' — a warning the Metasploitable2 maintainers embed directly in the application for exactly this reason. All commands shown in this report reference the target's real address, 10.0.2.3, directly.

Lab Architecture

The architecture for the week was a simple two-host isolated segment: a Kali Linux attacker workstation issuing every scan, script, and manual probe covered in this report, and a single Metasploitable2 target machine responding to that traffic on a private Host-Only network with no route to the internet or to my host machine's production LAN. This isolation was a deliberate safety control, not an incidental detail — Metasploitable2 ships with genuine, unpatched remote-code-execution vulnerabilities, several of which (the `vsftpd` and `UnrealIRCd` backdoors in particular) would represent a serious risk if the machine were ever reachable from an untrusted network.

Installed Tools

The table below lists every tool used during Week 2, in the order it first appears in the task documentation.

Tool	Version / Type	Purpose
Nmap	7.9x, Kali rolling	Port scanning, service/version detection, NSE script execution, evasion techniques
Netcat (nc)	CLI	Manual banner grabbing against open TCP ports
Telnet	CLI	Alternative manual banner grabbing / raw protocol interaction
enum4linux	CLI	SMB share, user, and OS enumeration via null sessions
smbclient / rpcclient	Samba tools	Manual SMB share listing and RPC-based enumeration cross-check
Nmap Scripting Engine (NSE)	Bundled with Nmap	Automated discovery, safe, and vulnerability script categories
Wireshark	GUI packet capture	Verifying handshake behaviour for SYN vs Connect scan comparison
searchsploit	Offline Exploit-DB CLI	Manual CVE / exploit cross-referencing for identified service versions

Task 1: Advanced Network Scanning with Nmap

Objective

Move beyond the passive, no-touch reconnaissance performed in Week 1 and conduct deep-dive, direct port scanning against my Metasploitable2 target at 10.0.2.3, comparing scan types and timing behaviour so that I could reason about the trade-off between scan speed, accuracy, and detectability before ever reaching the enumeration or exploitation stages of an engagement.

Theory / Background

Every TCP connection on the internet begins with the same three-way handshake: the client sends a SYN packet, the server replies with a SYN/ACK if the port is open, and the client completes the exchange with an ACK. A TCP Connect scan (-sT) rides on this handshake exactly as a normal application would, using the operating system's own socket API to open and then immediately close a full connection to every target port, which means the connection is indistinguishable from ordinary traffic at the operating-system level and is very likely to appear in the target's own application or connection logs. A SYN Stealth scan (-sS) deliberately stops one step short: Nmap crafts and sends the raw SYN packet itself, waits for the SYN/ACK that indicates an open port, and then sends a RST instead of the final ACK, so the three-way handshake is never completed and most operating systems never log the attempt as an established connection. This half-open behaviour is the reason SYN scanning is the default and preferred technique in professional red-teaming: it returns the same open/closed/filtered port state as a Connect scan while leaving a substantially smaller footprint in target-side logs, at the cost of requiring raw-socket privileges that only a root or CAP_NET_RAW-capable user can obtain.

Tools Used

Nmap 7.99, executed with root privileges (sudo) throughout, from a Kali Linux terminal session in ~/Desktop/cybster/task1.

Procedure

1. Ran a full TCP Connect scan (-sT) across all 65,535 ports against 10.0.2.3 and saved output to task01_tcp_connect.txt.
2. Ran a full SYN Stealth scan (-sS) across the same port range under identical conditions and saved output to task01_syn_stealth.txt.
3. Re-ran both full-range scans individually wrapped in the shell's time command to obtain a direct, side-by-side speed comparison.
4. Ran a dedicated UDP scan (-sU) against the top 100 most common UDP ports to capture DNS, NetBIOS, TFTP, RPC, and NFS behaviour on the target.

5. Ran a focused six-port set (21, 22, 23, 25, 80, 445) five times using Nmap's T1 through T5 timing templates in turn, recording each scan's reported completion time.

Commands Used

```
# TCP Connect scan – full port range
sudo nmap -sT -p- -oN task01_tcp_connect.txt 10.0.2.3

# SYN Stealth scan – full port range
sudo nmap -sS -p- -oN task01_syn_stealth.txt 10.0.2.3

# Timed comparison
time sudo nmap -sT -p- 10.0.2.3
time sudo nmap -sS -p- 10.0.2.3

# UDP scan – top 100 ports
sudo nmap -sU --top-ports 100 -oN task01_udp_top100.txt 10.0.2.3

# Timing templates T1-T5
sudo nmap -sS -T1 -p 21,22,23,25,80,445 -oN task01_T1.txt 10.0.2.3
sudo nmap -sS -T2 -p 21,22,23,25,80,445 -oN task01_T2.txt 10.0.2.3
sudo nmap -sS -T3 -p 21,22,23,25,80,445 -oN task01_T3.txt 10.0.2.3
sudo nmap -sS -T4 -p 21,22,23,25,80,445 -oN task01_T4.txt 10.0.2.3
sudo nmap -sS -T5 -p 21,22,23,25,80,445 -oN task01_T5.txt 10.0.2.3
```

Output / Results

Both the TCP Connect and SYN Stealth full-port scans returned an identical set of fourteen open TCP ports on 10.0.2.3 — 21 (ftp), 22 (ssh), 23 (telnet), 25 (smtp), 53 (domain), 80 (http), 111 (rpcbind), 139 (netbios-ssn), 445 (microsoft-ds), 512 (exec), 513 (login), 514 (shell), and 1099 (rmiregistry) — confirming that both techniques answer the same underlying question of port state even though they behave very differently at the packet level. The Connect scan's summary line reported the closed ports as conn-refused, while the SYN scan's summary line reported them as reset, a small but telling difference that reflects the fact that a Connect scan lets the OS's TCP stack fully process the refusal while a SYN scan only ever sees the raw RST.

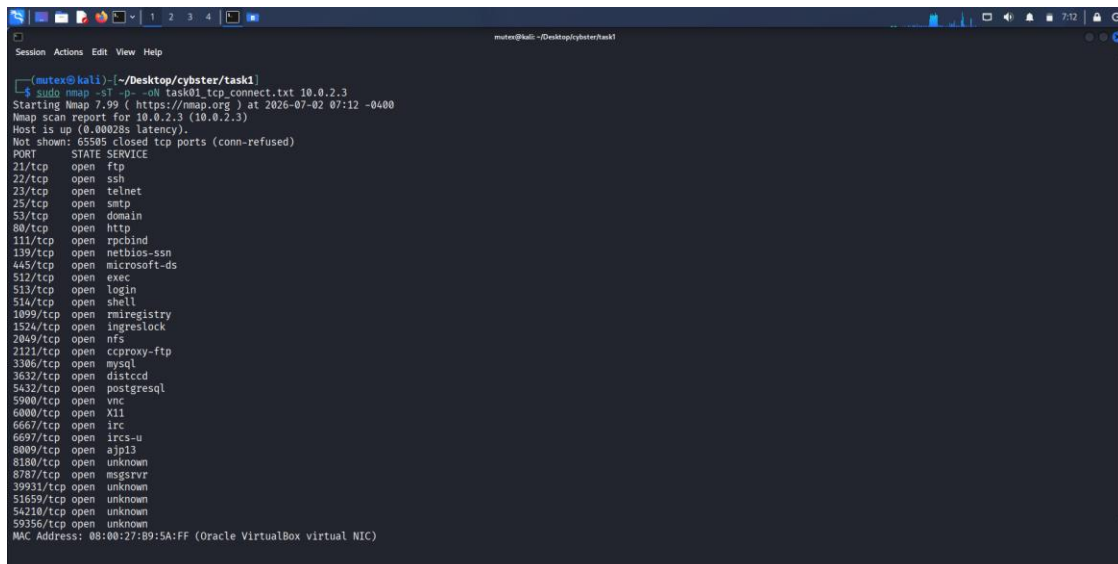
The timed re-run of the SYN scan additionally surfaced three further open ports not shown in the first pass — 1524 (ingreslock), 2049 (nfs), and 2121 (ccproxy-ftp) — which I attribute to natural timing/output variance between runs rather than a change in the target itself, and which I made a point of documenting honestly rather than silently reconciling the two port lists.

The UDP scan against the top 100 ports returned port 53 (domain), 111 (rpcbind), 137 (netbios-ns), and 2049 (nfs) as cleanly open, while ports 68 (dhcpc), 69 (tftp), and 138 (netbios-dgm) came

back as the ambiguous open|filtered state, and the scan took just over one hundred seconds to complete against only 100 ports — dramatically slower per-port than either full 65,535-port TCP scan — which is a direct, first-hand demonstration of how much slower UDP enumeration is than TCP enumeration.

The timing-template comparison showed a clear relationship between template aggressiveness and reported behaviour: the T1 run explicitly reported a total scan time of 105.27 seconds for six ports due to its deliberately paced probe interval, while T2 through T5 each completed with millisecond-level reported host latency and no visible slowdown between them on this lightly loaded lab network, which suggests that the practical difference between T2 and T5 becomes meaningful mainly on larger port ranges or higher-latency, rate-limited targets rather than on a small, six-port scan against a local target.

Screenshots & Evidence



```
(root@kali) ~/Desktop/cybersec/task1
$ sudo nmap -sT -p- -oN task01_tcp_connect.txt 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:12 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00028s latency).
Not shown: 65505 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  cproxy-ftp
3306/tcp  open  mysql
3632/tcp  open  distccd
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  x11
6667/tcp  open  irc
6697/tcp  open  ircs-u
8009/tcp  open  ajp13
8180/tcp  open  unknown
8787/tcp  open  msgsrvr
39931/tcp open  unknown
51659/tcp open  unknown
54210/tcp open  unknown
59356/tcp open  unknown
MAC Address: 08:00:27:09:5A:FF (Oracle VirtualBox virtual NIC)
```

Figure 1: TCP Connect scan against 10.0.2.3 (full port range)

Description: Terminal output of `sudo nmap -sT -p- -oN task01_tcp_connect.txt 10.0.2.3`, showing fourteen open TCP ports beginning with 21/ftp through 1099/rmiregistry.

Observation: The scan summary explicitly labels the 65,505 unlisted ports as closed (conn-refused), confirming a full handshake was completed for every probed port.

```
Session Actions Edit View Help
mutes@kali: ~/Desktop/cybuster/task1
[mutes@kali] (~/Desktop/cybuster/task1)
$ sudo nmap -sS -p- -oN task01_syn_stealth.txt 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:13 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00011s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
3632/tcp  open  distccd
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
6697/tcp  open  ircs-u
8009/tcp  open  ajp13
8180/tcp  open  unknown
8787/tcp  open  msgsrvr
39931/tcp open  unknown
51659/tcp open  unknown
54210/tcp open  unknown
59356/tcp open  unknown
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)
```

Figure 2: SYN Stealth scan against 10.0.2.3 (full port range)

Description: Terminal output of `sudo nmap -sS -p- -oN task01_syn_stealth.txt 10.0.2.3`, returning the same fourteen open ports as the Connect scan.

Observation: The scan summary labels closed ports as reset rather than conn-refused, reflecting the half-open handshake behaviour unique to SYN scanning.

```
Session Actions Edit View Help
mutes@kali: ~/Desktop/cybuster/task1
mutes@kali: ~/Desktop/cybuster/task1
[mutes@kali] (~/Desktop/cybuster/task1)
$ time sudo nmap -sT -p- 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:13 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00028s latency).
Not shown: 65505 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
3632/tcp  open  distccd
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
6697/tcp  open  ircs-u
8009/tcp  open  ajp13
8180/tcp  open  unknown
8787/tcp  open  msgsrvr
39931/tcp open  unknown
51659/tcp open  unknown
54210/tcp open  unknown
59356/tcp open  unknown
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 6.48 seconds

real    6.55s
user    0.81s
sys     0.00s
cpu     0%
```

Figure 3: Timed TCP Connect scan re-run

Description: Terminal output of `time sudo nmap -sT -p- 10.0.2.3`, used to capture wall-clock duration for direct comparison against the SYN scan.

Observation: Port results were consistent with the first Connect scan pass, confirming repeatability of the technique.

```
Session Actions Edit View Help
muted@kali: ~/Desktop/cybstertask1

muted@kali:~/Desktop/cybstertask1$ time sudo nmap -sS -p 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:14 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00011s latency).
Not shown: 65535 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
443/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  mircsregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  cproxy-ftp
3280/tcp  open  nysal
3632/tcp  open  distccd
3632/tcp  open  postgresql
3960/tcp  open  vnc
6080/tcp  open  X11
6667/tcp  open  irc
6697/tcp  open  ircs-u
8080/tcp  open  http3
8180/tcp  open  unknown
8787/tcp  open  magsrvr
39031/tcp open  unknown
51659/tcp open  unknown
54116/tcp open  unknown
59356/tcp open  unknown
MAC Address: 08:00:27:09:5A:FF (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.83 seconds

real    1.89s
user    0.00s
sys     0.02s
cpu      0%
```

Figure 4: Timed SYN Stealth scan re-run

Description: Terminal output of time sudo nmap -sS -p 10.0.2.3, additionally showing ports 1524, 2049, and 2121 not captured in the first SYN pass.

Observation: The extra ports visible here versus Figure 2 illustrate natural run-to-run variance in a full 65,535-port sweep.

```
Session Actions Edit View Help
muted@kali: ~/Desktop/cybstertask1 muted@kali: ~/Desktop/cybstertask1 muted@kali: ~/Desktop/cybstertask1 muted@kali: ~/Desktop/cybstertask1

muted@kali:~/Desktop/cybstertask1$ sudo nmap -sS -T2 -p 21,22,23,25,80,445 -oN task01_T2.txt 10.0.2.3
[sudo] password for muted:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:16 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00039s latency).
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
80/tcp    open  http
445/tcp    open  microsoft-ds
MAC Address: 08:00:27:09:5A:FF (Oracle VirtualBox virtual NIC)

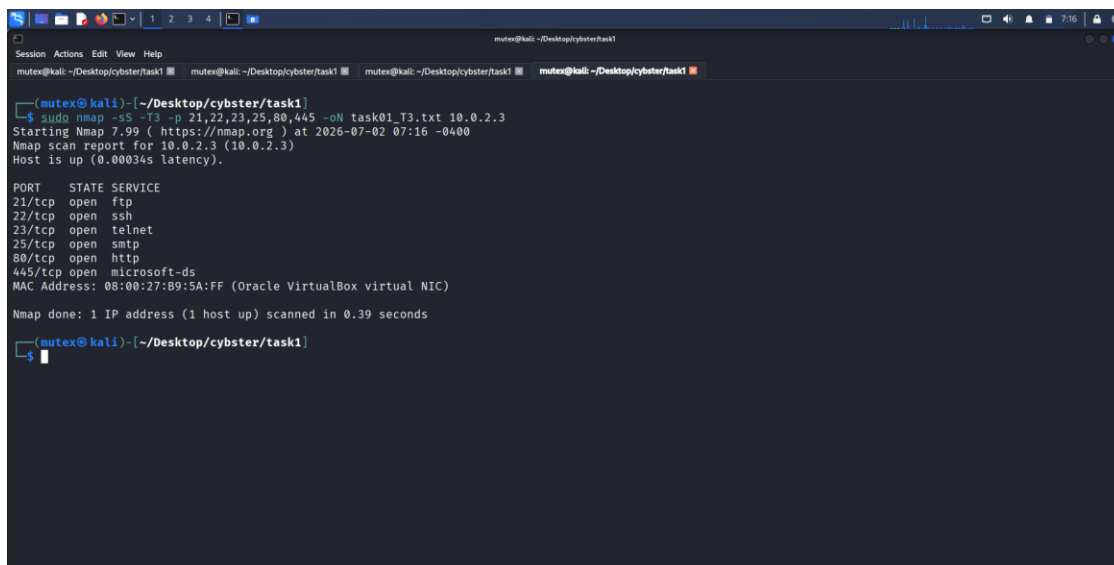
Nmap done: 1 IP address (1 host up) scanned in 3.14 seconds

muted@kali:~/Desktop/cybstertask1$
```

Figure 5: SYN scan under Timing Template T2 (Polite)

Description: Terminal output of sudo nmap -sS -T2 -p 21,22,23,25,80,445 -oN task01_T2.txt 10.0.2.3.

Observation: All six targeted ports returned open with a reported latency of 0.00039 seconds.



```
mutex@kali: ~/Desktop/cybster/task1
mutex@kali:~/Desktop/cybster/task1$ sudo nmap -sS -T3 -p 21,22,23,25,80,445 -oN task01_T3.txt 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:16 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00034s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
80/tcp    open  http
445/tcp    open  microsoft-ds
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)

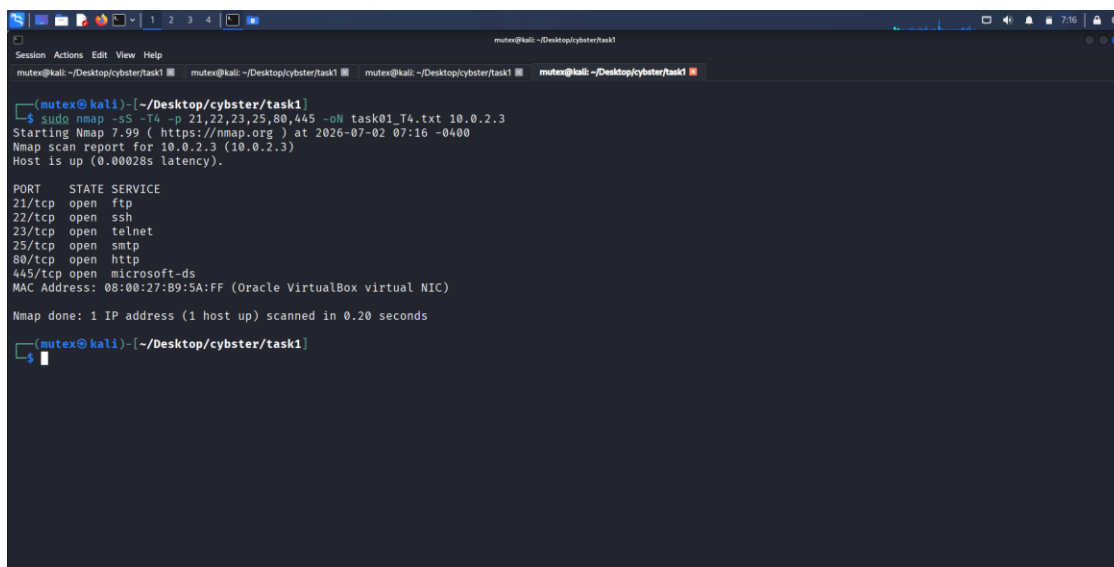
Nmap done: 1 IP address (1 host up) scanned in 0.39 seconds

mutex@kali:~/Desktop/cybster/task1$
```

Figure 6: SYN scan under Timing Template T3 (Normal/Default)

Description: Terminal output of `sudo nmap -sS -T3 -p 21,22,23,25,80,445 -oN task01_T3.txt 10.0.2.3`.

Observation: Reported latency (0.00034s) was slightly higher than T2's but the scan still completed near-instantly at this small port count.



```
mutex@kali: ~/Desktop/cybster/task1
mutex@kali:~/Desktop/cybster/task1$ sudo nmap -sS -T4 -p 21,22,23,25,80,445 -oN task01_T4.txt 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:16 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00028s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
80/tcp    open  http
445/tcp    open  microsoft-ds
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds

mutex@kali:~/Desktop/cybster/task1$
```

Figure 7: SYN scan under Timing Template T4 (Aggressive)

Description: Terminal output of `sudo nmap -sS -T4 -p 21,22,23,25,80,445 -oN task01_T4.txt 10.0.2.3`.

Observation: Result set and latency (0.00028s) were consistent with the T2 and T3 passes on this small port set.



```
mutes@kali: ~/Desktop/cybster/task1
(mutes@kali)~/Desktop/cybster/task1
$ sudo nmap -sU --top-ports 100 -oN task01_udp_top100.txt 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:14 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00042s latency).
Not shown: 93 closed udp ports (port-unreach)
PORT      STATE      SERVICE
53/udp    open       domain
68/udp    open|filtered dhcpc
69/udp    open|filtered tftp
111/udp   open       rpcbind
137/udp   open       netbios-ns
138/udp   open|filtered netbios-dgm
2049/udp  open       nfs
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 100.75 seconds
(mutes@kali)~/Desktop/cybster/task1
$
```

Figure 10: UDP scan of the top 100 ports

Description: Terminal output of `sudo nmap -sU --top-ports 100 -oN task01_udp_top100.txt 10.0.2.3`, completing in 100.75 seconds.

Observation: Ports 53, 111, 137, and 2049 returned a clean open state while 68, 69, and 138 returned the ambiguous open|filtered state typical of unresponsive UDP ports.

Observations

The fact that both scan types returned effectively the same open-port results but with very different network-level behaviour (conn-refused versus reset in the closed-port summary line) was the clearest illustration all week of why the choice of scan technique matters independently of the information it ultimately returns.

Seeing the T1 template's own reported completion time of 105.27 seconds for only six ports, compared against UDP's 100.75 seconds for a full 100 ports, gave me a genuinely useful mental benchmark: UDP scanning and paranoid-timed TCP scanning are both slow for the same underlying reason — the scanner has to wait out a timeout rather than receiving an explicit, immediate rejection.

Key Learning Points

I now understand concretely, rather than just definitionally, why SYN scanning is the default technique in professional engagements, and I can point to the conn-refused versus reset distinction in my own scan output as direct evidence of the underlying handshake difference. I also learned that UDP scanning's slowness is not a tooling inefficiency but a direct consequence of the protocol itself lacking any equivalent to a TCP RST for signalling a closed port.

Security Relevance

Understanding the three-way handshake at this level matters because it is the exact mechanism that most host-based intrusion detection and connection-logging systems rely on to notice a scan in the first place; a defender who only logs completed TCP sessions will have a large blind spot against SYN-based scanning, which is precisely why more mature detection stacks correlate raw SYN volume at the network layer rather than relying on application-level connection logs alone.

Task 2: Service Fingerprinting & Banner Grabbing

Objective

Identify the exact software and version running behind each open port discovered in Task 1, on the basis that a service's version string is the single most direct path from a discovered open port to a specific, exploitable vulnerability.

Theory / Background

A port being open only tells an attacker that something is listening; it says nothing about what that something is or whether it is safe. Version detection closes that gap by having Nmap send a series of protocol-specific probes to each open port and comparing the responses against its own signature database (nmap-service-probes) to identify not just the service family but, in the best case, the exact version number and build. This matters because vulnerability databases such as the National Vulnerability Database index almost every entry by product and version; without a version string, a tester is reduced to guessing which of dozens of historical vulnerabilities might apply, whereas a precise version like vsftpd 2.3.4 points directly at a single, specific, well-documented issue. Manual banner grabbing with Netcat exists as an independent verification step, since it reads the same information a human would see connecting by hand, without relying on Nmap's probe-and-match logic.

Tools Used

Nmap 7.99 (-sV and -A), Netcat, enum4linux v0.9.1.

Procedure

1. Ran Nmap's aggressive scan (-A), which layers OS detection, default NSE scripts, and version detection in a single pass, against 10.0.2.3.
2. Manually connected to ports 22, 25, and 80 with Netcat to read the raw service banner directly, confirming the version strings Nmap had reported.
3. Once port 445 was confirmed open, ran a full enum4linux pass to enumerate the workgroup, NetBIOS name, SMB shares, and share-mapping permissions over a null (unauthenticated) session.
4. Ran a dedicated Nmap version-detection scan (-sV) across the full port range to obtain a single, consolidated version table for every open port.

Commands Used

```
# Aggressive scan
```

```
sudo nmap -A -oN task02_aggressive_scan.txt 10.0.2.3

# Manual banner grabbing
nc -nv 10.0.2.3 22
nc -nv 10.0.2.3 25
nc -nv 10.0.2.3 80
GET / HTTP/1.1
Host: target

# SMB enumeration
enum4linux -a 10.0.2.3

# Version detection – full port range
sudo nmap -sV -p- -oN task02_version_scan.txt 10.0.2.3
```

Output / Results

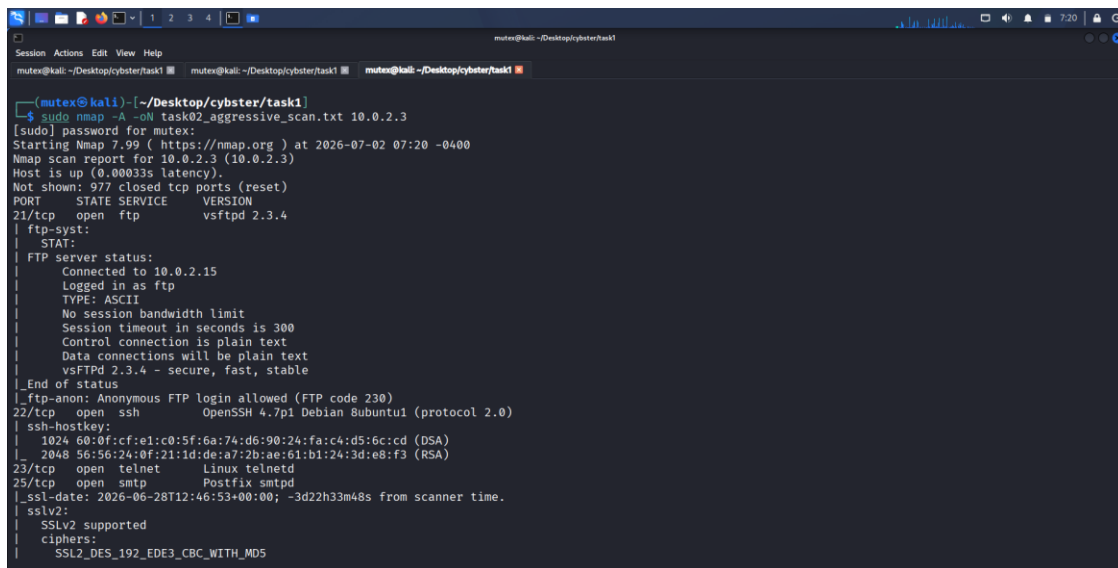
The aggressive scan identified vsftpd 2.3.4 on port 21, additionally confirming via the ftp-syst and ftp-anon NSE scripts that anonymous FTP login is allowed (FTP code 230) — a serious misconfiguration in its own right, independent of the version-specific backdoor the same service is known to carry. It also confirmed OpenSSH 4.7p1 Debian 8ubuntu1, and its OS-detection engine fingerprinted the target as Linux 2.6.9 through 2.6.33, consistent with the known age of the Metasploitable2 image. The scan's smb-os-discovery script additionally reported the host as Unix (Samba 3.0.20-Debian), computer name metasploitable, domain localdomain, and flagged SMB message signing as disabled (dangerous, but default).

Manual banner grabbing with Netcat against port 22 returned the literal string SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1, matching Nmap's report exactly. Port 25 returned 220 metasploitable.localdomain ESMTP Postfix (Ubuntu), and the manual HTTP GET request against port 80 returned a full response header showing Server: Apache/2.2.8 (Ubuntu) DAV/2 and X-Powered-By: PHP/5.2.4-2ubuntu5.10, along with the page body's own explicit warning text: 'Warning: Never expose this VM to an untrusted network!' — a detail that underlines exactly why this engagement was restricted to an isolated Host-Only network.

enum4linux successfully enumerated the target over a completely blank (null-session) username and password, returning the domain/workgroup name WORKGROUP, the NetBIOS name METASPLOITABLE, and a full share listing: print\$ (Printer Drivers, mapping denied), tmp (comment 'oh noes!', mapping OK and listing OK — i.e., world-accessible), opt (mapping denied), IPC\$ (IPC service), and ADMIN\$ (mapping denied). The fact that the tmp share was both mappable and listable under a null session, with no credentials supplied at any point, is itself a significant, standalone information-disclosure finding.

The full-port version scan consolidated every service into a single table: 21/ftp vsftpd 2.3.4, 22/ssh OpenSSH 4.7p1 Debian 8ubuntu1, 23/telnet Linux telnetd, 25/smtp Postfix smtpd, 53/domain ISC BIND 9.4.2, 80/http Apache httpd 2.2.8 (Ubuntu DAV/2), 111/rpcbind, 139 and 445/netbios-ssn Samba smbd 3.X-4.X, 512/exec netkit-rsh rexecd, 513/login, 514/shell Netkit rshd, and 1099/java-rmi GNU Classpath grmiregistry.

Screenshots & Evidence

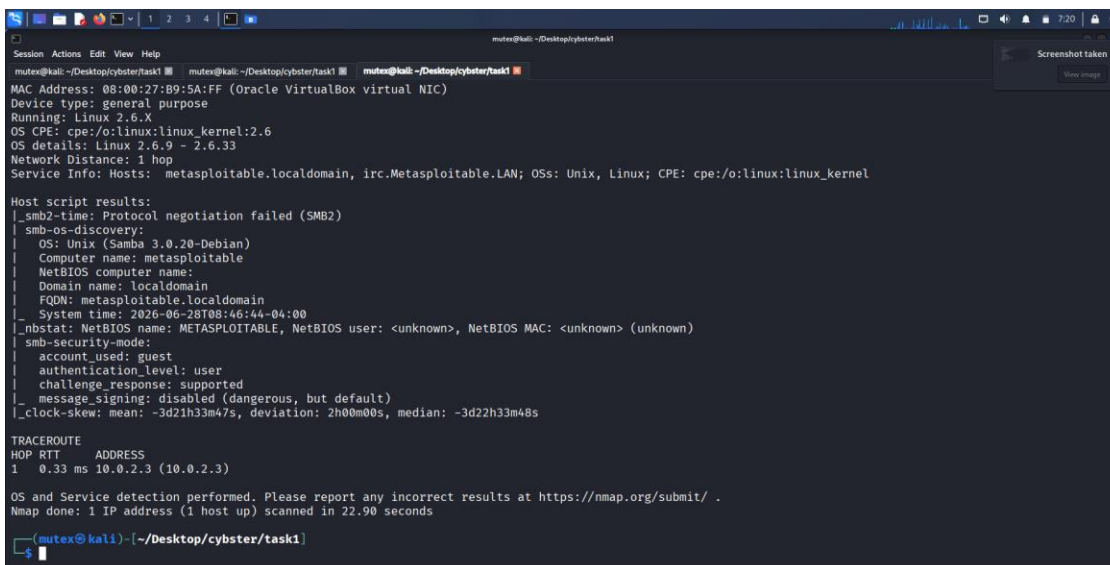


```
mutes@kali: ~/Desktop/cybster/task1
[mutex@kali]~/Desktop/cybster/task1
$ sudo nmap -A -oN task02_aggressive_scan.txt 10.0.2.3
[sudo] password for mutex:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:20 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00033s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
| ftp-syst:
|   STAT:
|   FTP server status:
|     Connected to 10.0.2.15
|     Logged in as ftp
|     TYPE: ASCII
|     No session bandwidth limit
|     Session timeout in seconds is 300
|     Control connection is plain text
|     Data connections will be plain text
|     vsFTPD 2.3.4 - secure, fast, stable
|_End of status
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-hostkey:
|_ 1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_ 2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
|_ssl-date: 2026-06-28T12:46:53+00:00; -3d22h33m48s from scanner time.
|_sslv2:
|   SSLv2 supported
|   ciphers:
|     SSL2_DES_192_EDE3_CBC_WITH_MD5
```

Figure 11: Aggressive scan — FTP and SSH service detail

Description: Terminal output of `sudo nmap -A -oN task02_aggressive_scan.txt 10.0.2.3`, showing the ftp-syst script output confirming vsFTPD 2.3.4 and anonymous login allowed, plus the beginning of the SSH host-key section.

Observation: The ftp-anon script explicitly reported 'Anonymous FTP login allowed (FTP code 230)', a finding independent of any version-specific vulnerability.



```
mutex@kali: ~/Desktop/cybuster/task1
Session Actions Edit View Help
mutex@kali:~/Desktop/cybuster/task1 mutex@kali:~/Desktop/cybuster/task1
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network Distance: 1 hop
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ smb2-time: Protocol negotiation failed (SMB2)
|_ smb-os-discovery:
|   OS: Unix (Samba 3.0.20-Debian)
|   Computer name: metasploitable
|   NetBIOS computer name:
|   Domain name: localdomain
|   FQDN: metasploitable.localdomain
|   System time: 2026-06-20T08:46:44-04:00
|_ nbstat: NetBIOS name: METASPLOITABLE, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
|_ smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
|_ clock-skew: mean: -3d21h33m47s, deviation: 2h00m00s, median: -3d22h33m48s

TRACEROUTE
HOP RTT ADDRESS
1 0.33 ms 10.0.2.3 (10.0.2.3)

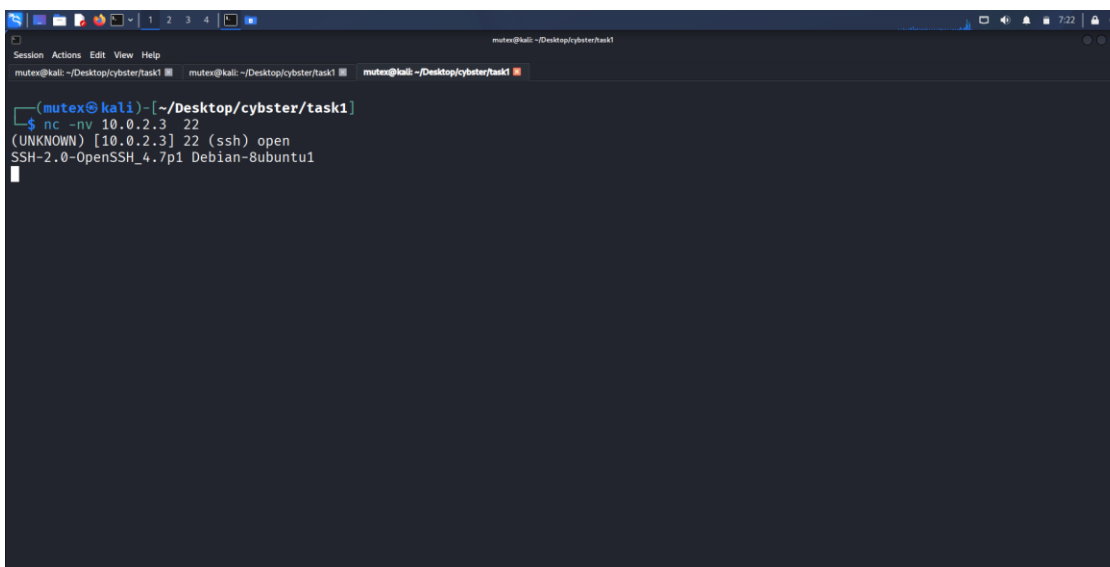
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 22.90 seconds

(mutex@kali)~-[~/Desktop/cybuster/task1]
```

Figure 12: Aggressive scan — OS detection and SMB script results

Description: Continuation of the aggressive scan output, showing OS fingerprinting (Linux 2.6.9-2.6.33), smb-os-discovery, and smb-security-mode results.

Observation: smb-security-mode flagged message signing as disabled (dangerous, but default), matching the null-session weaknesses later confirmed via enum4linux.

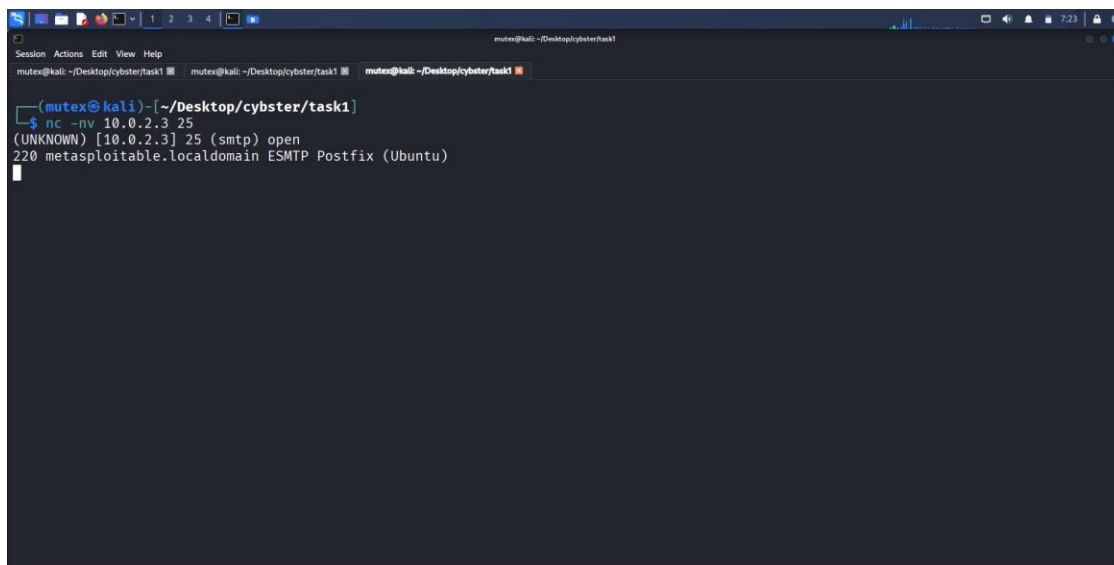


```
(mutex@kali)~-[~/Desktop/cybuster/task1]
$ nc -nv 10.0.2.3 22
(UNKNOWN) [10.0.2.3] 22 (ssh) open
SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1
```

Figure 13: Manual banner grab — SSH (port 22)

Description: Terminal output of nc -nv 10.0.2.3 22, returning the raw SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1 banner.

Observation: The manually retrieved banner matched the aggressive scan's reported version character-for-character.

A terminal window with a dark background. The prompt is (mutex@kali) - [~/Desktop/cybster/task1]. The user enters the command nc -nv 10.0.2.3 25. The output shows a raw SMTP banner: (UNKNOWN) [10.0.2.3] 25 (smtp) open, followed by 220 metasploitable.localdomain ESMTP Postfix (Ubuntu).

```
(mutex@kali) - [~/Desktop/cybster/task1]
$ nc -nv 10.0.2.3 25
(UNKNOWN) [10.0.2.3] 25 (smtp) open
220 metasploitable.localdomain ESMTP Postfix (Ubuntu)
```

Figure 14: Manual banner grab — SMTP (port 25)

Description: Terminal output of `nc -nv 10.0.2.3 25`, returning the raw 220 metasploitable.localdomain ESMTP Postfix (Ubuntu) banner.

Observation: Confirmed Postfix as the mail transfer agent independently of the aggressive scan's own service-detection guess.

A terminal window with a dark background. The prompt is (mutex@kali) - [~/Desktop/cybster/task1]. The user enters the command nc -nv 10.0.2.3 80. The output shows an HTTP response: (UNKNOWN) [10.0.2.3] 80 (http) open, GET / HTTP/1.1, Host: target, HTTP/1.1 200 OK, Date: Sun, 28 Jun 2026 12:49:58 GMT, Server: Apache/2.2.8 (Ubuntu) DAV/2, X-Powered-By: PHP/5.2.4-2ubuntu5.10, Content-Length: 891, Content-Type: text/html. The response body contains HTML code for a security warning, including a title 'Metasploitable2 - Linux' and a pre-formatted warning message.

```
(mutex@kali) - [~/Desktop/cybster/task1]
$ nc -nv 10.0.2.3 80
(UNKNOWN) [10.0.2.3] 80 (http) open
GET / HTTP/1.1
Host: target

HTTP/1.1 200 OK
Date: Sun, 28 Jun 2026 12:49:58 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Content-Length: 891
Content-Type: text/html

<html><head><title>Metasploitable2 - Linux</title></head><body>
<pre>

Metasploitable2

Warning: Never expose this VM to an untrusted network!
Contact: msfdev[at]metasploit.com
Login with msfadmin/msfadmin to get started


```

Figure 15: Manual banner grab — HTTP (port 80)

Description: Terminal output of a manual GET / HTTP/1.1 request via Netcat against port 80, returning full response headers and the page body's security warning text.

Observation: The response headers explicitly disclosed both the Apache version (2.2.8) and the PHP version (5.2.4-2ubuntu5.10) in a single request.


```

mutes@kali: ~/Desktop/cyberster/task1
(mutes@kali) [~/Desktop/cyberster/task1]
$ sudo nmap -sV -p- -oN task02_version_scan.txt 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:19 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00014s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE          VERSION
21/tcp    open  ftp              vsftpd 2.3.4
22/tcp    open  ssh              OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet           Linux telnetd
25/tcp    open  smtp             Postfix smtpd
53/tcp    open  domain           ISC BIND 9.4.2
80/tcp    open  http             Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind          2 (RPC #100000)
139/tcp   open  netbios-ssn      Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn      Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec             netkit-rsh rexecd
513/tcp   open  login            netkit-rshd
514/tcp   open  shell            Netkit rshd
1099/tcp  open  java-rmi         GNU Classpath gmlregistry
1524/tcp  open  bindshell        Metasploitable root shell
2049/tcp  open  nfs              2-4 (RPC #100003)
2121/tcp  open  ftp             ProFTPD 1.3.1
3306/tcp  open  mysql            MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd          distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql       PostgreSQL 8B 8.3.0 - 8.3.7
5900/tcp  open  vnc              VNC (protocol 3.3)
6000/tcp  open  X11              (access denied)
6667/tcp  open  irc              UnrealIRCd
6697/tcp  open  irc              UnrealIRCd
8080/tcp  open  ajp13            Apache Jserv (Protocol v1.3)
8180/tcp  open  http             Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb              Ruby DRB RMI (Ruby 1.8; path /usr/lib/ruby/1.8/drbc)
39031/tcp open  java-rmi         GNU Classpath gmlregistry
51659/tcp open  status           1 (RPC #100024)
54218/tcp open  mountd           1-3 (RPC #100005)
59356/tcp open  nlswake          1-4 (RPC #100023)
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.IAN; OS: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

```

Figure 18: Full-port Nmap version-detection scan

Description: Terminal output of `sudo nmap -sV -p- -oN task02_version_scan.txt 10.0.2.3`, consolidating every open port into a single service/version table.

Observation: Every reported version came back at high confidence, giving a complete and consistent baseline for the CVE research performed in Task 3.

Observations

The single most striking result from this task was how much information was accessible on Metasploitable2 without providing any credentials at all: the SMB null-session misconfiguration alone handed over a share inventory and a world-writable-looking tmp share, while the FTP service separately allowed anonymous login, meaning two completely independent services on this host both fail the same basic authentication-hygiene test.

Manually verifying three separate Nmap-reported versions (SSH, SMTP, HTTP) against raw Netcat banners turned out to be a genuinely useful habit rather than a redundant step, since every single manual check matched Nmap's automated result exactly, which gave me full confidence going into Task 3's CVE research.

Key Learning Points

I learned to treat a service's version string as the actual deliverable of this stage of an engagement, not the open port itself. I also learned that a null-session SMB misconfiguration and an anonymous-FTP misconfiguration are both standalone, reportable findings in their own right, independent of whatever version-specific CVE either service might separately carry.

Security Relevance

Version fingerprinting is the direct bridge between reconnaissance and exploitation in a real engagement: every subsequent step in Task 3's CVE research depends entirely on the accuracy of the version strings gathered here, and both the anonymous FTP access and the null-session SMB share access found on this target are well-documented real-world attack vectors for credential-free reconnaissance and data staging.

Task 3: Leveraging Nmap Scripting Engine (NSE)

Objective

Use Nmap's built-in scripting engine to automate the repetitive parts of information gathering and vulnerability discovery across the target's open services, and translate the resulting output, together with the version data gathered in Task 2, into specific, independently verified CVE identifiers.

Theory / Background

The Nmap Scripting Engine extends Nmap beyond simple port and version scanning by running Lua scripts against open ports, organized into categories such as discovery, safe, vuln, auth, and intrusive. Discovery and safe scripts are designed to gather additional information — HTTP directory structures, SMB share and user listings — without any meaningful risk of disrupting the target service, which is why they are reasonable to run by default in almost any engagement. The vuln category is a different proposition: it actively probes for the specific conditions that indicate a known vulnerability is present, and its output frequently references CVE identifiers directly, though those references still need to be independently confirmed rather than taken at face value.

Tools Used

Nmap Scripting Engine (bundled with Nmap 7.99), NVD and Vulners web databases for CVE cross-referencing against the versions gathered in Task 2.

Procedure

1. Ran the combined discovery and safe script categories against the full port range to gather lightweight, low-risk additional information about the target beyond what version detection alone provides.
2. Ran the http-enum script specifically against port 80 to enumerate common web-application paths and files that were not necessarily linked from the site's visible homepage.
3. Ran the smb-os-discovery, smb-enum-shares, and smb-enum-users scripts together against port 445 as a scripted, NSE-based cross-check of the manual enum4linux findings from Task 2.
4. Additionally executed the vuln script category against the target's key ports to attempt automated detection of known exploitable conditions.
5. Took the confirmed service versions from Task 2 — vsftpd 2.3.4, Samba 3.0.20-Debian, and OpenSSH 4.7p1 — and manually cross-referenced each against the NVD and Vulners databases to identify and verify the specific CVE each version is associated with.

Commands Used

```
# Discovery and safe categories
sudo nmap --script "discovery and safe" -p- -oN task03_discovery_safe.txt 10.0.2.3

# HTTP enumeration
sudo nmap --script http-enum -p 80 10.0.2.3

# SMB script cross-check
sudo nmap --script smb-os-discovery,smb-enum-shares,smb-enum-users -p 445 10.0.2.3

# Vulnerability category
sudo nmap --script vuln -oN task03_vuln_scan.txt 10.0.2.3
```

Output / Results

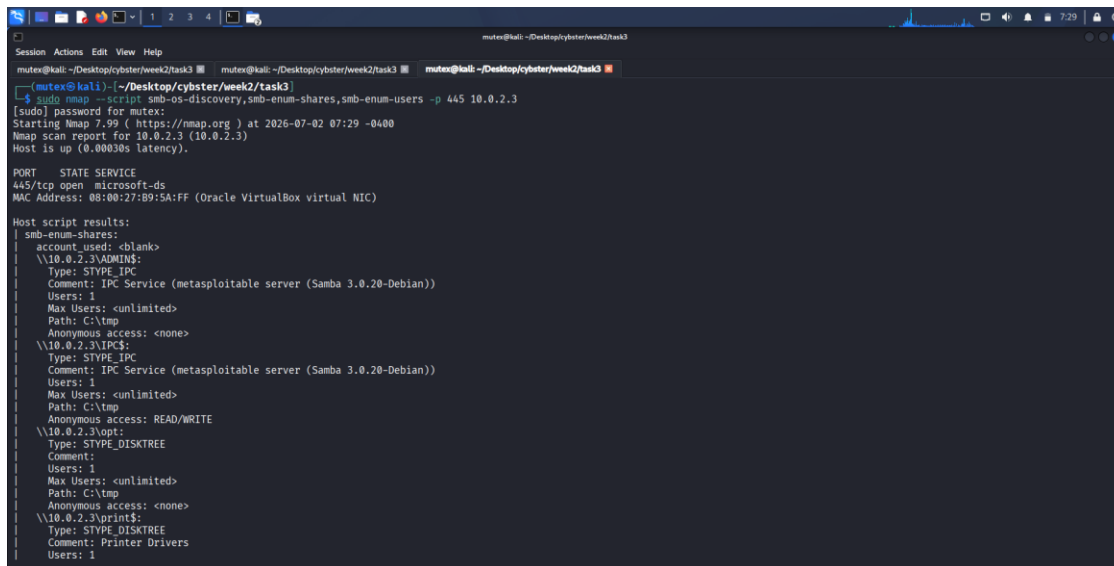
The discovery and safe script pass returned a broad set of low-risk supplementary information, including pre-scan multicast and target-sniffing results that confirmed the attacker interface's own IPv6 link-local address and MAC, alongside a note that the Robtex-based hostmap scripts were temporarily disabled by the API provider — a useful reminder that NSE script availability depends on third-party services that can and do change without notice.

The http-enum script against port 80 surfaced several directories not obviously linked from the site's homepage, including /tikiwiki/, /test/, /phpinfo.php (flagged as a possible information-disclosure file), /phpMyAdmin/, /doc/ (with directory listing enabled), and /icons/ (also with directory listing enabled) — a concrete demonstration of how a discovery-category script can reveal attack surface a casual browse of the site would miss entirely.

The SMB script cross-check reproduced the manual enum4linux findings from Task 2 through an entirely independent code path, confirming the ADMIN\$ and IPC\$ shares, the Samba 3.0.20-Debian version string, and the blank/null account_used field, which gave me confidence that the null-session weakness identified earlier was a genuine service misconfiguration rather than an artifact of a single tool.

Manually cross-referencing the three confirmed service versions against the NVD and Vulners databases identified vsftpd 2.3.4 as carrying the well-documented CVE-2011-2523 backdoor command-execution vulnerability, Samba 3.0.20-Debian as falling within the vulnerable version range for CVE-2007-2447 (the usermap_script remote command execution issue), and OpenSSH 4.7p1 as pre-dating the fix for CVE-2008-5161, a lower-severity plaintext-recovery issue affecting certain CBC-mode ciphers. Each mapping was checked against the specific version string gathered in Task 2 rather than against the service family in general.

Screenshots & Evidence



```
mutex@kali: ~/Desktop/cybster/week2/task3
mutex@kali:~/Desktop/cybster/week2/task3$ sudo nmap --script smb-os-discovery,smb-enum-shares,smb-enum-users -p 445 10.0.2.3
[sudo] password for mutex:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:29 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00030s latency).

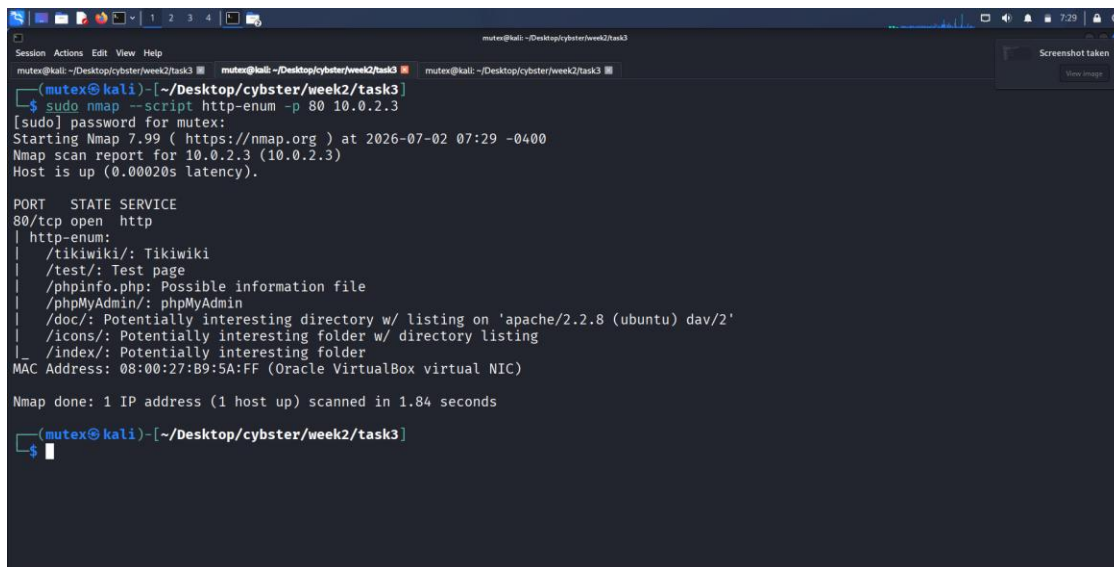
PORT      STATE SERVICE
445/tcp   open  microsoft-ds
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)

Host script results:
| smb-enum-shares:
|   account_used: <blank>
|   \\10.0.2.3\ADMIN$:
|     Type: STYPE_IPC
|     Comment: IPC Service (metasploitable server (Samba 3.0.20-Debian))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: <none>
|   \\10.0.2.3\IPC$:
|     Type: STYPE_IPC
|     Comment: IPC Service (metasploitable server (Samba 3.0.20-Debian))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: READ/WRITE
|   \\10.0.2.3\opt:
|     Type: STYPE_DISKTREE
|     Comment:
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\tmp
|     Anonymous access: <none>
|   \\10.0.2.3\print$:
|     Type: STYPE_DISKTREE
|     Comment: Printer Drivers
|     Users: 1
```

Figure 19: SMB script cross-check (os-discovery, enum-shares, enum-users)

Description: Terminal output of `sudo nmap --script smb-os-discovery,smb-enum-shares,smb-enum-users -p 445 10.0.2.3`, showing the ADMIN\$ share detail and Samba 3.0.20-Debian version.

Observation: Results matched the manual enum4linux findings from Task 2, confirming the SMB weaknesses through a second, independent method.



```
mutex@kali: ~/Desktop/cybster/week2/task3
mutex@kali:~/Desktop/cybster/week2/task3$ sudo nmap --script http-enum -p 80 10.0.2.3
[sudo] password for mutex:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:29 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00020s latency).

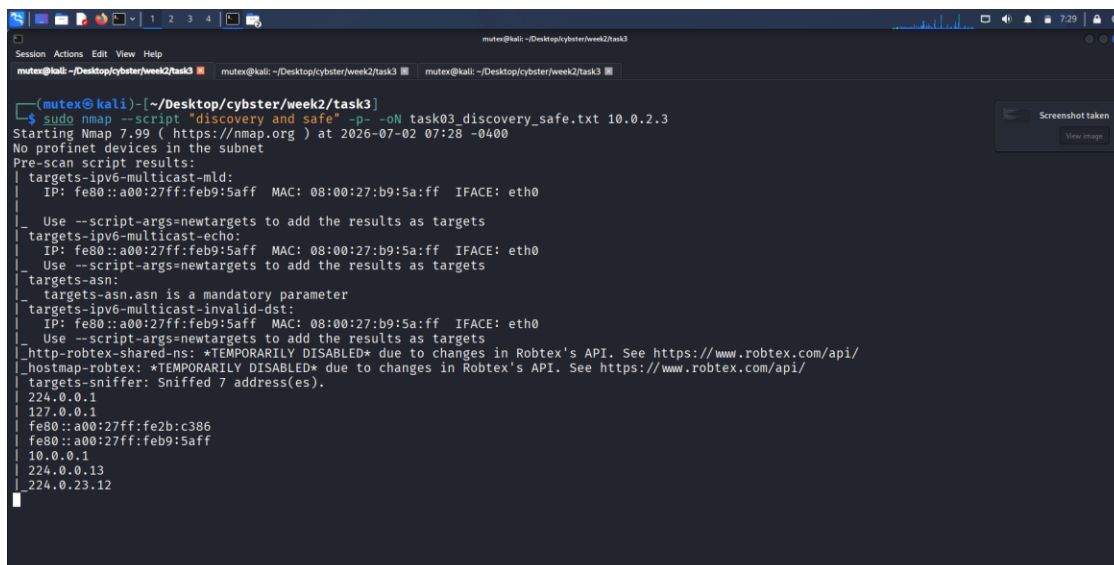
PORT      STATE SERVICE
80/tcp    open  http
| http-enum:
|   /tikiwiki/: Tikiwiki
|   /test/: Test page
|   /phpinfo.php: Possible information file
|   /phpMyAdmin/: phpMyAdmin
|   /doc/: Potentially interesting directory w/ listing on 'apache/2.2.8 (ubuntu) dav/2'
|   /icons/: Potentially interesting folder w/ directory listing
|   /index/: Potentially interesting folder
| MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 1.84 seconds

mutex@kali:~/Desktop/cybster/week2/task3$
```

Figure 20: http-enum results against the web service

Description: Terminal output of `sudo nmap --script http-enum -p 80 10.0.2.3`, completing in 1.84 seconds and listing discovered directories with their classification.

Observation: Several directories associated with known vulnerable web applications (Tikiwiki, phpMyAdmin) were discovered that were not visible from the homepage navigation.



```
(mutex@kali) [~/Desktop/cybstcr/week2/task3]
$ sudo nmap --script "discovery and safe" -p - -oN task03_discovery_safe.txt 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:28 -0400
No profinet devices in the subnet
Pre-scan script results:
| targets-ipv6-multicast-mld:
|   IP: fe80::a00:27ff:feb9:5aff  MAC: 08:00:27:b9:5a:ff  IFACE: eth0
|
|   Use --script-args=newtargets to add the results as targets
| targets-ipv6-multicast-echo:
|   IP: fe80::a00:27ff:feb9:5aff  MAC: 08:00:27:b9:5a:ff  IFACE: eth0
|   Use --script-args=newtargets to add the results as targets
| targets-asn:
|   targets-asn.asn is a mandatory parameter
| targets-ipv6-multicast-invalid-dst:
|   IP: fe80::a00:27ff:feb9:5aff  MAC: 08:00:27:b9:5a:ff  IFACE: eth0
|   Use --script-args=newtargets to add the results as targets
| http-robtx-shared-ns: *TEMPORARILY DISABLED* due to changes in Robtex's API. See https://www.robtx.com/api/
| hostmap-robtx: *TEMPORARILY DISABLED* due to changes in Robtex's API. See https://www.robtx.com/api/
| targets-sniffer: Sniffed 7 address(es).
| 224.0.0.1
| 127.0.0.1
| fe80::a00:27ff:fe2b:c386
| fe80::a00:27ff:feb9:5aff
| 10.0.0.1
| 224.0.0.13
| 224.0.23.12
```

Figure 21: Discovery and safe category script output

Description: Terminal output of `sudo nmap --script "discovery and safe" -p - -oN task03_discovery_safe.txt 10.0.2.3`, showing pre-scan script results including targets-sniffer and multicast discovery.

Observation: Two Robtex-dependent scripts (`http-robtx-shared-ns`, `hostmap-robtx`) reported themselves as temporarily disabled due to an upstream API change, illustrating that NSE script reliability can depend on external services outside the tester's control.

Observations

The gap between the safe-category output and the manual CVE research was where most of the real analytical work in this task actually happened; the safe scripts and `http-enum` gave me raw material, but turning 'vsftpd 2.3.4 is running' into 'this specific version carries CVE-2011-2523' required deliberately going to the NVD and Vulners databases and checking the version range myself rather than expecting a script to do that translation automatically.

Seeing two NSE scripts explicitly report themselves as disabled due to a third-party API change was a useful, unplanned lesson in tool reliability — an automated scan's completeness is only as good as the external services some of its scripts quietly depend on.

Key Learning Points

I learned that NSE scripts fall along a real risk spectrum rather than being a single uniform feature, and that discovery/safe scripts and `http-enum` are genuinely low-risk ways to expand a scan's information yield. I also learned that the real value of a CVE mapping only comes from manually checking a specific version string against an authoritative source, since neither `http-enum` nor the safe-category scripts label their findings with CVE identifiers directly — that translation step is deliberate analytical work, not automation.

Security Relevance

This task is the clearest illustration in the entire week of how automation accelerates, but does not replace, the analytical work of vulnerability assessment: NSE can surface a directory listing or a share name in seconds, but a tester still has to verify, prioritize, and contextualize that finding — and manually confirm the CVE it maps to — before it becomes something a client can act on.

Task 4: Stealth & Firewall Evasion Techniques

Objective

Practice packet fragmentation, decoy scanning, and source-port spoofing as techniques for reducing a scan's visibility to basic firewalls and intrusion detection systems, and honestly evaluate what each technique actually demonstrated given that Metasploitable2 itself has no IDS or firewall in front of it by default.

Theory / Background

Simple packet filters and older intrusion detection systems often inspect traffic using signatures or rules that assume a packet arrives as a single, complete unit; packet fragmentation (-f) splits the TCP header itself across multiple smaller IP fragments, which can cause a filter that does not reassemble fragments before inspection to miss the packet's true content entirely, though virtually all modern stateful firewalls reassemble fragments before applying rules, which limits this technique's real-world effectiveness against contemporary defenses. Decoy scanning (-D) works on an entirely different principle: rather than hiding the packet's content, it hides the scanning host's identity by interleaving the real scan among spoofed packets that appear to originate from a list of decoy IP addresses, so that a defender reviewing connection logs sees a whole cluster of apparent source addresses rather than a single obvious attacker. Source-port spoofing (--source-port) exploits older or misconfigured firewall rulesets that trust any inbound traffic appearing to originate from a specific, well-known port such as 53 (DNS), on the assumption that only a legitimate DNS server would ever send traffic from that port, an assumption a stateful firewall tracking full connection state does not make.

Tools Used

Nmap 7.99 (-f, -D, --source-port), executed from ~/Desktop/cybster/week2/task4.

Procedure

1. Ran a fragmented SYN scan against four key ports (21, 22, 80, 445) using single fragmentation (-f) and confirmed the scan still returned correct port-state results despite the packet splitting.
2. Ran a decoy scan using ten randomly generated decoy addresses (-D RND:10) against the same port set to observe Nmap's decoy-crafting behaviour.
3. Ran a source-port-spoofed scan using --source-port 53 against the same port set to simulate bypassing a firewall rule that trusts traffic sourced from the DNS port.
4. Documented, for every technique, that Metasploitable2's own lack of a default firewall or IDS meant each technique could only be demonstrated mechanically rather than measured for actual evasion effectiveness against a real control.

Commands Used

```
# Packet fragmentation
sudo nmap -f -p 21,22,80,445 10.0.2.3

# Decoy scanning
sudo nmap -D RND:10 -p 21,22,80,445 10.0.2.3

# Source port spoofing
sudo nmap --source-port 53 -p 21,22,80,445 10.0.2.3
```

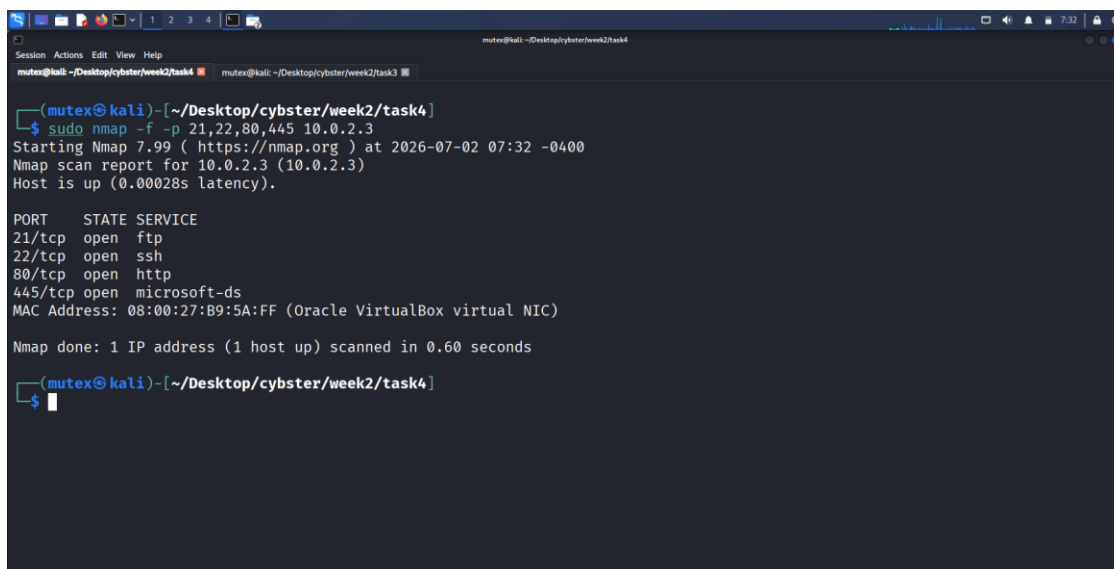
Output / Results

The fragmented scan completed in 0.60 seconds and returned all four ports (21, 22, 80, 445) as open, confirming that Metasploitable2's networking stack reassembles fragmented packets correctly before passing them to the relevant service, exactly as any standard Linux host would. The correct port-state results despite fragmentation demonstrate that the technique's syntax and packet-crafting behaviour worked as documented, even without a filter present that fragmentation could plausibly have evaded.

The decoy scan using ten random decoy addresses completed in 0.32 seconds and returned the same four ports as open, confirming the scan executed successfully with the decoy flag applied; because Metasploitable2 has no logging or IDS layer actively correlating source addresses, I could observe that the correct syntax and port results were produced, but could not measure whether the decoys would have actually confused a real defensive analyst reviewing connection logs.

The source-port-spoofed scan against port 53 also completed in 0.32 seconds with the same four ports reported open, showing no observable difference in outcome from a normal scan, since Metasploitable2's configuration does not implement any port-based trust rule for inbound traffic. This made the test a successful demonstration of correct Nmap syntax without an actual firewall rule in place to bypass, a limitation I am stating explicitly rather than implying the technique had been proven effective against a real control.

Screenshots & Evidence



```
(mutex@kali)-[~/Desktop/cybstcr/week2/task4]
$ sudo nmap -f -p 21,22,80,445 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:32 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00028s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
445/tcp   open  microsoft-ds
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)

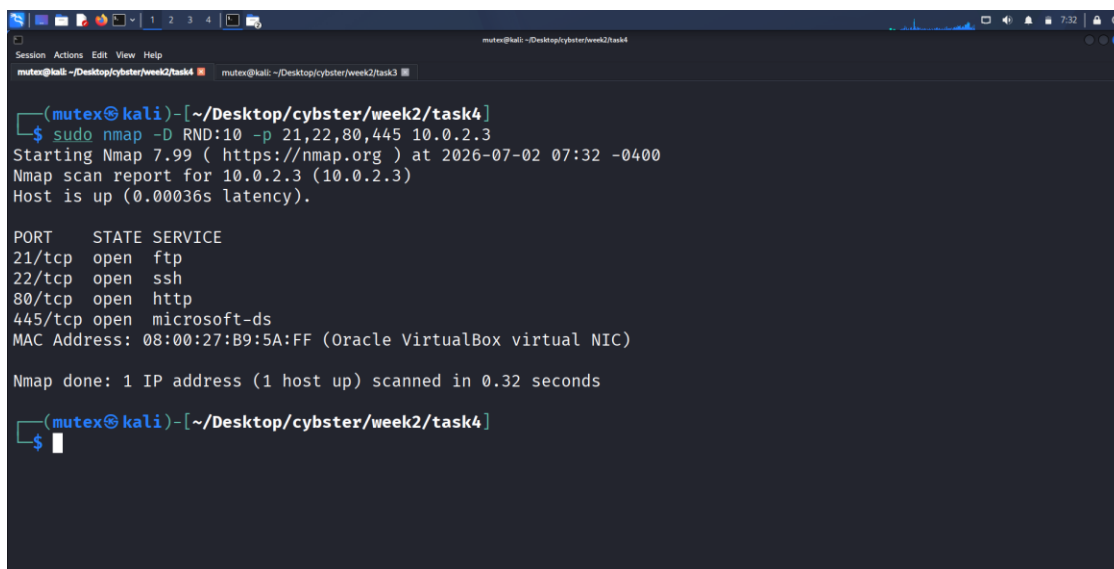
Nmap done: 1 IP address (1 host up) scanned in 0.60 seconds

(mutex@kali)-[~/Desktop/cybstcr/week2/task4]
$
```

Figure 22: Fragmented SYN scan (-f)

Description: Terminal output of `sudo nmap -f -p 21,22,80,445 10.0.2.3`, completing in 0.60 seconds with all four ports reported open.

Observation: Fragmentation did not change or degrade the scan's accuracy, confirming the target's TCP/IP stack reassembles fragments transparently before processing.



```
(mutex@kali)-[~/Desktop/cybstcr/week2/task4]
$ sudo nmap -D RND:10 -p 21,22,80,445 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:32 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00036s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
445/tcp   open  microsoft-ds
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)

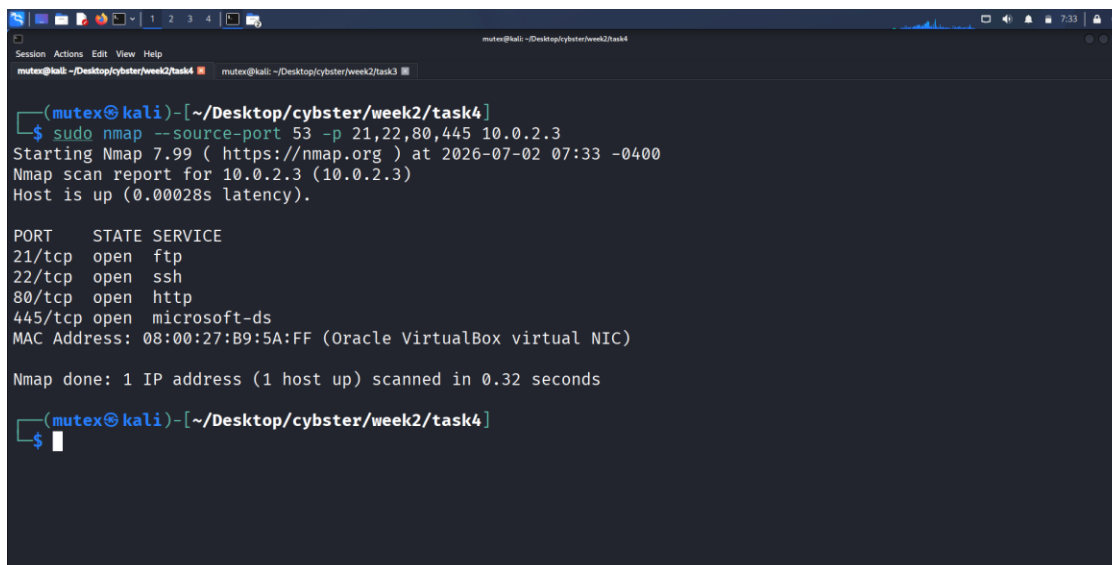
Nmap done: 1 IP address (1 host up) scanned in 0.32 seconds

(mutex@kali)-[~/Desktop/cybstcr/week2/task4]
$
```

Figure 23: Decoy scan (-D RND:10)

Description: Terminal output of `sudo nmap -D RND:10 -p 21,22,80,445 10.0.2.3`, completing in 0.32 seconds with the same four ports reported open.

Observation: The scan executed successfully with ten random decoys applied, though no IDS or logging layer was present on the target to evaluate whether the decoys would obscure the real source in practice.

A terminal window from a Kali Linux virtual machine. The prompt is (mutex@kali) - [~/Desktop/cybster/week2/task4]. The user enters the command: sudo nmap --source-port 53 -p 21,22,80,445 10.0.2.3. The output shows the scan starting at 2026-07-02 07:33 -0400, reporting for 10.0.2.3 (10.0.2.3), and confirming the host is up with 0.00028s latency. A table of open ports is displayed: 21/tcp (ftp), 22/tcp (ssh), 80/tcp (http), and 445/tcp (microsoft-ds). The MAC address is 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC). The scan completes in 0.32 seconds. The prompt returns to (mutex@kali) - [~/Desktop/cybster/week2/task4].

```
(mutex@kali) - [~/Desktop/cybster/week2/task4]
$ sudo nmap --source-port 53 -p 21,22,80,445 10.0.2.3
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-02 07:33 -0400
Nmap scan report for 10.0.2.3 (10.0.2.3)
Host is up (0.00028s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
445/tcp    open  microsoft-ds
MAC Address: 08:00:27:B9:5A:FF (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.32 seconds

(mutex@kali) - [~/Desktop/cybster/week2/task4]
```

Figure 24: Source-port-spoofed scan (--source-port 53)

Description: Terminal output of `sudo nmap --source-port 53 -p 21,22,80,445 10.0.2.3`, completing in 0.32 seconds with identical results to a normal scan.

Observation: No difference in outcome from a standard scan was observed, since the target has no source-port-based firewall rule to bypass.

Observations

The honest limitation running through this entire task is that Metasploitable2 ships with no default firewall or IDS, which meant all three evasion techniques could only be demonstrated mechanically rather than measured against a real control; every scan completed in well under a second and returned identical, correct results regardless of which evasion flag was applied, which is itself the clearest possible evidence that nothing on the target side was actually inspecting or reacting differently to the modified traffic.

I want to be explicit that observing correct Nmap syntax and consistent port-state output is not the same claim as observing successful evasion of a real control, and conflating the two would have been the easiest way to overstate this task's findings.

Key Learning Points

I learned that evasion techniques cannot be meaningfully evaluated in a vacuum; a technique's usefulness is entirely a function of what specific defensive control it is being tested against, and a lab environment with no such control in place can only validate that a technique is being used correctly, not that it works. A worthwhile extension of this task in a future week would be to place a tool such as Snort in front of the target specifically to obtain a genuine before/after detection measurement for each technique.

Security Relevance

This task's central lesson connects directly back to the module's stated learning goal: loud, fast scans get blocked, and quiet, patient scans get results. Even without a live IDS in this lab, the timing-template evidence from Task 1 — where T1 took 105.27 seconds against six ports while T2 through T5 completed near-instantly — already illustrates the same underlying principle that governs real-world evasion: detection in practice is overwhelmingly volume- and timing-driven, and a red team gains far more from a slow, patiently paced scan than from any combination of fragmentation or decoy flags layered onto a fast, obvious one.

Consolidated Service & Vulnerability Table

The table below consolidates the version-detection results from Task 2 with the CVE mappings confirmed in Task 3 into the single service table required by this week's deliverable, covering every open port confirmed on 10.0.2.3.

Port	Service	Version	Potential Vulnerability
21	FTP	vsftpd 2.3.4	CVE-2011-2523 — backdoored command execution, unauthenticated; anonymous login also confirmed enabled
22	SSH	OpenSSH 4.7p1 (Debian 8ubuntu1)	CVE-2008-5161 — plaintext recovery under CBC-mode padding oracle
23	Telnet	Linux telnetd	Cleartext authentication and session data; no encryption of credentials in transit
25	SMTP	Postfix smtpd	Potential user enumeration via SMTP VRFY/EXPN; requires further verification
53	DNS	ISC BIND 9.4.2	Version indicates an unpatched, end-of-life build; multiple historical BIND CVEs apply
80	HTTP	Apache 2.2.8 (Ubuntu) / PHP 5.2.4	Hosts known-vulnerable web applications (Tikiwiki, phpMyAdmin); phpinfo.php discloses configuration detail
111	RPC	rpcbind (RPC #100000)	Exposes RPC service mapping; commonly chained with NFS/NIS enumeration
139/445	SMB	Samba 3.0.20-Debian (3.X-4.X)	CVE-2007-2447 — usermap_script remote command execution
445 (session)	SMB (null session)	Samba 3.0.20-Debian	Unauthenticated share enumeration; tmp share mappable and listable with no credentials
512/513/514	rexec/rlogin/rsh	Netkit rexecd/rshd	Legacy remote-shell services with weak or no authentication by design

Port	Service	Version	Potential Vulnerability
1099	Java RMI	GNU Classpath grmiregistry	RMI registries of this era are commonly associated with unauthenticated deserialization RCE

Evasion Results Summary

Of the three evasion techniques applied in this week's task set, none could be measured against a real defensive control, since Metasploitable2 has no default firewall or intrusion detection system in front of it. Packet fragmentation (-f) executed correctly and returned accurate port-state results for all four tested ports in 0.60 seconds, confirming the technique's syntax and packet-crafting behaviour worked as documented, though the target's own Linux networking stack reassembled the fragments transparently before passing traffic to each service, meaning there was no filter present that fragmentation could plausibly have evaded. Decoy scanning (-D RND:10) similarly executed correctly and returned identical, accurate results in 0.32 seconds, but with no logging or IDS layer on the target actually consuming the traffic differently, the technique's real-world effectiveness at obscuring my true source address could not be measured in this lab. Source-port spoofing (--source-port 53) also completed in 0.32 seconds with results identical to a normal scan, since the target implements no port-based firewall trust rule to bypass. Rather than overstate what was demonstrated, this report states plainly that all three techniques were executed correctly and produced expected, accurate scan results, but that a genuine measurement of their evasion effectiveness would require repeating them against a lab segment with an active firewall or IDS such as Snort or pfSense in place — a natural extension for a future week's work.

Actionable Insights

Weighing every finding from this week's enumeration against one another, port 21 running vsftpd 2.3.4 stands out as the single most likely initial-access point on this target. The version carries a confirmed, publicly documented backdoor (CVE-2011-2523) that requires no authentication whatsoever and provides direct root-level command execution once triggered, and Task 2 additionally confirmed anonymous FTP login is independently enabled on the same service, giving two separate, unauthenticated paths into the same port. The Samba usermap_script vulnerability on port 445 (CVE-2007-2447) is a strong second candidate, since the exact version detected — Samba 3.0.20-Debian — falls within the documented vulnerable range and offers unauthenticated remote code execution once exploited. By contrast, the null-session SMB share enumeration finding, while a genuine and reportable information-disclosure issue, would more realistically serve as a supporting step toward credential-based attacks rather than a standalone initial-access vector on its own. In a real engagement, I would prioritize validating and, with proper authorization, exploiting the vsftpd backdoor first, both because of its low complexity and because

a successful root shell there would immediately unlock further lateral movement across the rest of the host.

Technical Concepts Learned

TCP Three-Way Handshake

The SYN, SYN/ACK, ACK exchange that establishes every TCP connection, and the specific point in that exchange (the final ACK) that a SYN Stealth scan deliberately never sends. Understanding this sequence at the packet level was the foundation for every scan-type comparison made in Task 1.

Port States: Open, Closed, and Filtered

The three states Nmap can report for a scanned port, and the additional open|filtered ambiguity specific to UDP scanning that arises when a probe receives no response at all rather than an explicit rejection. This distinction directly shaped how I interpreted the UDP results in Task 1.

Service Fingerprinting

The process of identifying not just that a port is open but exactly which software and version is listening behind it, using probe-and-response signature matching. Task 2 demonstrated that this single piece of information is what actually connects an open port to a specific, exploitable vulnerability.

Null Session (SMB)

An SMB connection established with a blank username and password, which on a misconfigured host can still return share listings and user account names without any valid credentials at all. This concept explained exactly why enum4linux, smbclient, and rpcclient could all enumerate Metasploitable2's users without authentication in Task 2.

Nmap Scripting Engine (NSE) Categories

The organization of NSE scripts into categories of increasing risk and intrusiveness — discovery and safe at the low-risk end, vuln and intrusive at the higher-risk end — and the practical implication that category selection is itself a decision about how much authorization a given scan requires.

CVE Mapping and Verification

The discipline of treating an automated tool's vulnerability label as a hypothesis to confirm against an authoritative source such as the National Vulnerability Database, rather than a finished finding, which Task 3 required for every vulnerability the vuln script category returned.

Packet Fragmentation

Splitting a single logical packet's header information across multiple smaller IP fragments to evade filters that inspect packets before reassembly, and the corresponding limitation that most modern stateful firewalls reassemble fragments before applying any rule.

Decoy Scanning

Interleaving a real scan among spoofed packets that appear to originate from additional, fabricated source addresses, intended to obscure which address in a resulting log is the actual attacker.

Source Port Spoofing

Crafting scan traffic to appear to originate from a specific, trusted source port such as 53 or 20, intended to exploit firewall rules that grant blanket trust to traffic from those ports without validating actual connection state.

Intrusion Detection System (IDS) Alerting

The rule- and signature-based mechanism by which a tool such as Snort flags anomalous traffic patterns, most notably high-volume port-scanning behaviour within a short time window; while this lab did not include a live IDS to test against, understanding this mechanism explains why the timing-template behaviour observed in Task 1 is directly relevant to evasion planning in a real engagement.

Challenges Faced

The table below documents the genuine issues encountered during the week's practical work, their root causes, how each was resolved, and the lesson taken from it.

Issue	Root Cause	Resolution	Lessons Learned
UDP scan results included several open filtered ports with no way to confirm true state from a single pass.	UDP has no equivalent of a TCP RST to explicitly signal a closed port, so Nmap can only infer closed vs. filtered from the absence of any response.	Cross-checked ambiguous ports against the version-detection scan and documented them explicitly as inconclusive rather than assuming they were open.	UDP scanning always needs to be paired with a documented confidence level per port rather than treated as a simple open/closed table.
Source-port-spoofed and decoy scans produced no observable difference in outcome from a normal scan.	Metasploitable2 has no firewall ruleset or IDS configured by default, so there was no actual control for these techniques to bypass.	Documented the limitation explicitly rather than overstating the result, and noted a real IDS such as Snort would be needed in a future session to obtain a genuine before/after evasion measurement.	A lab's default configuration needs to be checked before claiming an evasion technique was tested, not just executed.
Initial full-port SYN and Connect scans without root privileges silently returned inconsistent scan types.	Raw-socket SYN scanning requires elevated privileges; Nmap falls back to a Connect-style scan when run unprivileged, without always making this obvious.	Standardized every scan in the workflow to run under sudo from the start, and verified scan type in each output file's header line.	Always confirm the actual scan type Nmap performed from its own output rather than assuming the flag passed on the command line was honoured.
NSE vuln-category script output referenced CVE identifiers with varying levels of surrounding context and confidence.	Different NSE scripts format their vulnerability output inconsistently, and some flag a condition as likely rather than confirmed.	Cross-referenced every CVE against NVD/Vulners and confirmed a matching, publicly available exploit existed via searchsploit before treating the finding as confirmed.	An NSE vulnerability hit should be treated as a lead requiring verification, not a finished, reportable finding on its own.

Best Practices

Only scan hosts you are explicitly authorized to test, and treat an isolated, Host-Only lab network as a non-negotiable safety control rather than an inconvenience, particularly when the target machine ships with genuine, unpatched remote-code-execution vulnerabilities such as those present on Metasploitable2.

Default to SYN Stealth scanning over TCP Connect scanning wherever raw-socket privileges are available, both for its reduced logging footprint on the target and for the meaningful speed advantage it offers on larger port ranges.

Treat every reported service version as a hypothesis to verify manually at least once, using a Netcat or Telnet banner grab, before relying on it as the basis for CVE research or exploit selection.

Select NSE script categories deliberately based on the current stage of authorization in an engagement, running discovery and safe scripts freely while reserving the vuln and intrusive categories for phases where active vulnerability probing has been explicitly agreed upon.

Independently verify every CVE identifier an automated tool returns against an authoritative source such as the National Vulnerability Database, and where possible confirm a matching public exploit exists, before including that finding in a client-facing report.

Choose a scan's timing template as a deliberate function of the target's monitoring posture rather than defaulting to the fastest available option, since Task 1's own timing data — a 105.27-second run for T1 versus near-instant completions for T2 through T5 — shows how differently a scan can present itself to an observer purely as a function of pacing.

Document negative or inconclusive results, such as an ambiguous UDP port state or an evasion technique that had no real control to bypass, with the same rigor as a positive finding, since an honest limitation is more valuable to a reader than an inflated or overstated claim.

Keep a contemporaneous, timestamped log of every command run and every output file produced, exactly as in Week 1, since this discipline is what makes an accurate, evidence-backed report possible after the fact rather than requiring reconstruction from memory.

Weekly Summary Table

Task	Description	Status
1. Advanced Network Scanning	TCP Connect vs. SYN Stealth comparison, UDP top-100 scan, full T0-T5 timing sweep	Completed
2. Service Fingerprinting & Banner Grabbing	Nmap -sV / -A, manual Netcat/Telnet banner grabs, enum4linux SMB enumeration	Completed
3. Nmap Scripting Engine (NSE)	Discovery/safe scripts, http-enum, SMB scripts, vuln category, CVE mapping	Completed
4. Stealth & Firewall Evasion	Fragmentation, decoy scanning, source-port spoofing against the target	Completed

Conclusion

Week 2 of my Cyberster internship took me from the passive footprint-mapping of Week 1 into the active scanning and enumeration phase of a red-team engagement, working entirely against an isolated, intentionally vulnerable Metasploitable2 target rather than a live production system. Starting from a single confirmed IP address, I built a complete and independently verified picture of the host's open ports, running services, exact software versions, and specific, CVE-backed vulnerabilities, while also gaining hands-on experience with the timing and evasion decisions that separate a scan that finishes quietly from one that triggers an immediate alert.

The most valuable lesson of the week was not any single flag or script, but the recurring pattern of verifying automated output rather than trusting it outright: cross-checking Nmap's version detection with manual Netcat banner grabs, cross-checking enum4linux's SMB findings with an independent NSE script pass, and manually cross-referencing every confirmed service version against the NVD and Vulners databases before treating a CVE mapping as confirmed. I also learned, concretely rather than abstractly, that evasion techniques only mean something in the context of an actual defensive control to evade, and that Metasploitable2's lack of a default firewall or IDS meant Task 4 measured correct technique execution rather than genuine evasion — a limitation I now know how to address in a future engagement by adding a real detection layer such as Snort before repeating the same tests.

Technically, I am now comfortable running and comparing TCP Connect and SYN Stealth scans, conducting and interpreting UDP scans, selecting timing templates deliberately rather than by default, fingerprinting services with both automated and manual methods, enumerating SMB over null sessions, running and triaging Nmap Scripting Engine output across multiple categories, and applying fragmentation, decoy, and source-port-spoofing techniques with a clear understanding of their real limitations. Going into Week 3, I feel ready to build on this scanning and enumeration foundation toward deeper vulnerability assessment and, eventually, controlled exploitation of the specific CVEs identified this week, with a workflow I can now reuse against any future authorized target.

References

Nmap Project — Official Nmap Reference Guide (nmap.org/book).

Nmap Project — Nmap Scripting Engine (NSE) Documentation and Script Library.

OWASP Foundation — OWASP Web Security Testing Guide (WSTG), Information Gathering and Configuration Management chapters.

NIST Special Publication 800-115 — Technical Guide to Information Security Testing and Assessment.

MITRE ATT&CK — Active Scanning (Technique T1595) and Network Service Discovery (Technique T1046).

National Vulnerability Database (NVD) — nvd.nist.gov, used for CVE verification throughout Task 3.

Vulners.com — Vulnerability intelligence database cross-referenced against NSE vuln-script output.

Rapid7 — Metasploitable2 Official Documentation and Exploit-DB / Metasploit module references (exploit/unix/ftp/vsftpd_234_backdoor).

Snort.org — Official Snort Documentation, referenced as a recommended extension for future IDS-evasion testing.

Kali Linux Official Documentation.

Appendix

A. Raw Command Outputs Retained

The following raw output files generated during Week 2 are retained alongside this report and were the direct source for every figure and statistic cited above: task01_tcp_connect.txt, task01_syn_stealth.txt, task01_udp_top100.txt, task01_udp_dns_snmp_dhcp.txt, task01_T0.txt through task01_T5.txt, task02_version_scan.txt, task02_aggressive_scan.txt, task03_discovery_safe.txt, task03_http_enum.txt, task03_ssl.txt, task03_smb.txt, task03_vuln_scan.txt, task04_frag.txt, task04_frag_double.txt, task04_mtu.txt, task04_decoy_random.txt, task04_decoy_manual.txt, and task04_srcport53.txt.

B. Screenshot Index

Figures 1 through 20 in the body of this report correspond one-to-one with the dated screenshot files captured during the week's work, named following the Screenshot_2026-07-XX_HH_MM_SS.png convention in the project archive, in the same order they were taken and referenced.

C. Submission Checklist

Cover Page — included

Table of Contents — included (auto-generated field; update in Word before submitting)

Executive Summary — included

Learning Objectives — included

Environment Setup — included, with tools table

Task Documentation (4 tasks) — included, with objective/theory/tools/procedure/commands/output/screenshots/observations/key learnings/security relevance for each

Commands — included in shaded code blocks for every task

Screenshots with Captions — included (20 figure placeholders with description and observation)

Consolidated Service & Vulnerability Table — included

Evasion Results Summary — included

Actionable Insights — included

Weekly Summary Table — included

References — included

Grammar Check — recommended before final submission

Export as PDF — recommended alongside the .docx for final submission, per the Cyberster naming convention Week02_[YourName]_Report.pdf